

Subiect 05 – Sistem de bibliotecă digitală

Cerințe obligatorii

1. Pattern-urile implementate trebuie să respecte definiția din GoF discutată în cadrul cursurilor și laboratoarelor. Nu sunt acceptate variații sau implementări incomplete.
2. Pattern-ul trebuie implementat în totalitate corect pentru a fi luat în calcul.
3. Soluția nu conține erori de compilare.
4. Pattern-urile pot fi tratate distinct sau pot fi implementate pe același set de clase.
5. Implementările care nu au legătura funcțională cu cerințele din subiect NU vor fi luate în calcul (preluare unui exemplu din alte surse nu va fi punctată).
6. NU este permisă modificare claselor primite.
7. Soluțiile vor fi verificate încrucișat folosind MOSS. Nu este permisă partajarea de cod între studenți. Soluțiile care au un grad de similitudine mai mare de 30% vor fi anulate.

Cerințe Clean Code obligatorii (soluția este depunctată cu câte 2 puncte pentru fiecare cerință ce nu este respectată) - maxim se pot pierde 8 puncte

1. Pentru denumirea claselor, funcțiilor, testelor unitare, atributelor și a variabilelor se respecta convenția de nume de tip Java Mix CamelCase;
2. Pattern-urile și clasa ce conține metoda main() sunt definite în pachete distincte ce au forma `cts.nume.prenume.gNrGrupa.denumire_pattern`, `cts.nume.prenume.gNrGrupa.main` (studenții din anul suplimentar trec "as" în loc de gNrGrupa);
3. Clasele și metodele sunt implementate respectând principiile KISS, DRY și SOLID (atenție la DIP);
4. Denumirile de clase, metode, variabile, precum și mesajele afișate la consola trebuie să aibă legătura cu subiectul primit (nu sunt acceptate denumiri generice). Funcțional, metodele vor afișa mesaje la consola care să simuleze acțiunea cerută sau vor implementa prelucrări simple.

Se dezvoltă o aplicație software destinată unei biblioteci digitale

3p. Se dorește accesarea unor documente digitale mari: cărți scanate, articole PDF și arhive multimedia. Încărcarea completă a unui document este costisitoare, iar unele documente pot fi accesate doar de utilizatori autentificați. Se cere introducerea unui obiect intermediar care să verifice drepturile de acces, să păstreze într-o structură internă documentele deja încărcate și să reutilizeze instanțele existente atunci când același document este cerut de mai multe ori. Pentru implementare se va folosi interfața *AbstractDocumentDigital*.

1.5p. Să se testeze soluția prin:

- accesarea aceluiași document de mai multe ori;
- verificarea accesului pentru utilizatori autorizați și neautorizați;
- afișarea numărului de documente reale încărcate;
- evidențierea reutilizării documentelor deja existente.

3p. Biblioteca permite organizarea colecțiilor sub formă de rafturi, categorii și subcategorii. O carte individuală are un număr de pagini, iar o categorie poate conține atât cărți, cât și alte categorii. Se dorește calcularea recursivă a numărului total de pagini și validarea faptului că o colecție nu depășește o limită maximă admisă. Pentru implementare se va folosi interfața *AbstractElementBiblioteca*.

1.5p. Să se testeze soluția prin:

- crearea mai multor cărți;
- gruparea lor în categorii și subcategorii;
- calcularea numărului total de pagini;
- verificarea unei limite maxime pentru o colecție complexă.

```
public interface AbstractDocumentDigital {  
    void afiseaza(String utilizator);  
    String getCod();  
}
```

```
public interface AbstractElementBiblioteca {  
    int calculeazaNumarPagini();  
    boolean valideazaLimita(int limitaMaxima);  
}
```